

Maritime CargoService: Sprint 1

Davide Chirichella, Gabriele Doti, Daniele
Maccagnan

Ingegneria dei Sistemi Software, A.A. 2025/2026
Alma Mater Studiorum . Università di Bologna

July 09, 2026

1. Introduction

Il punto di partenza di questo sprint è l'architettura logica (recuperabile al seguente [link](#)^o) e la formalizzazione dei requisiti (recuperabile al seguente [link](#)^o).

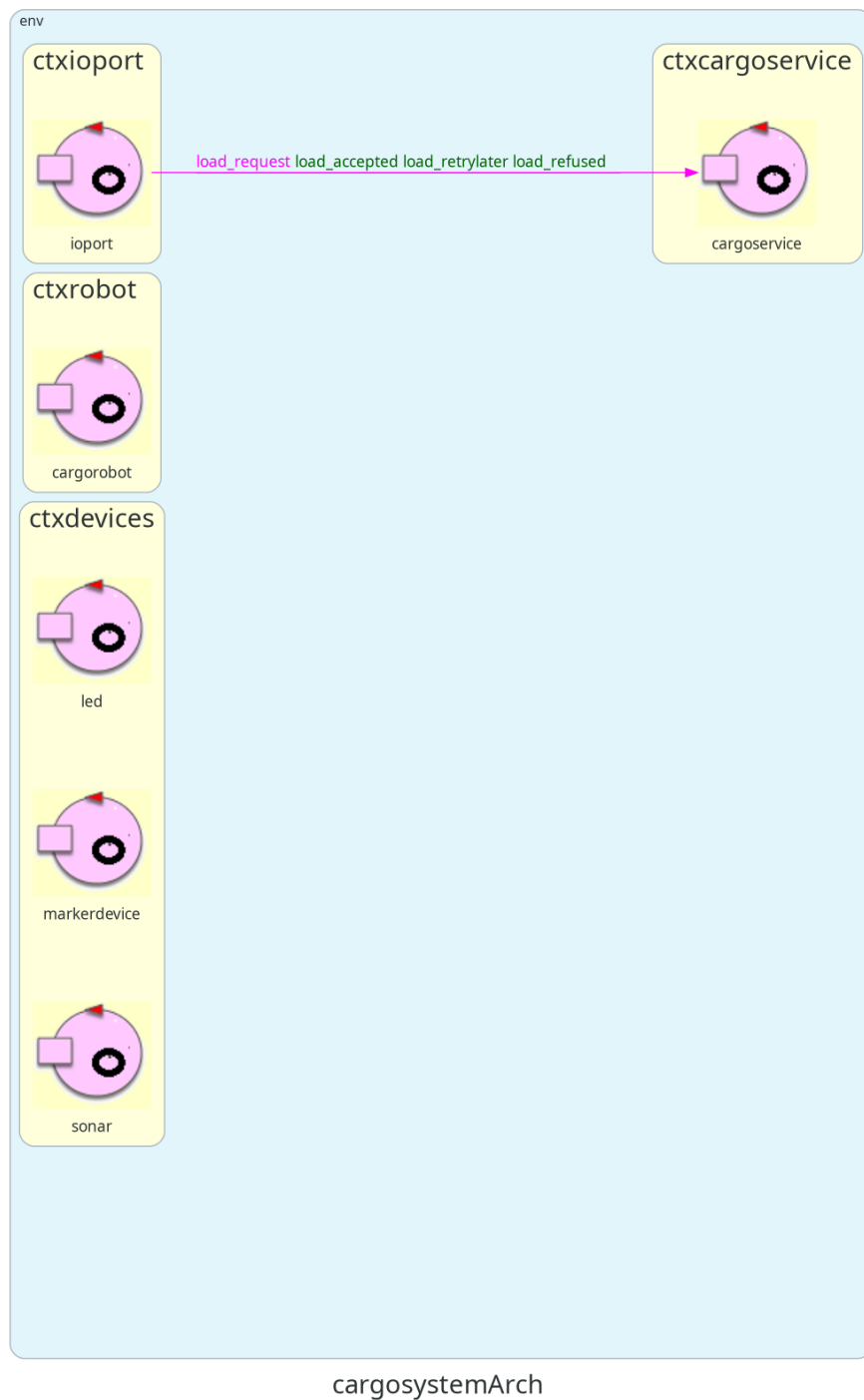


Figure 1: Architettura definita nello Sprint0.

Si riporta di seguito il goal dello sprint 1:

DA AGGIORNARE

Realizzare un primo prototipo eseguibile del **cargoservice** che ne implementi il comportamento descritto dai requisiti mediante collaboratori simulati. Poiché l'obiettivo principale è validare la logica del servizio, il movimento del robot viene astratto come collaborazione simulata, assumendo che l'operazione di deposito termini con esito positivo. Questa scelta consente di verificare prima il coordinamento tra i componenti e rimandare a una fase successiva il controllo fisico del robot e la gestione dei percorsi.

Al termine dello Sprint1 il sistema dovrà essere in grado di:

- ricevere una **load_request** proveniente dall'IOPort;
- verificare lo stato della hold, dell'IOPort e del servizio;
- produrre una delle tre risposte previste:
 - **load_accepted(slotID)**;
 - **load_retrylater**;
 - **load_refused**;
- mantenere il modello logico della hold e la prenotazione dello slot;
- gestire gli stati **engaged** e **disengaged**;
- pilotare il LED in funzione dello stato del sistema;
- superare i test funzionali definiti nello Sprint0.

In questa fase il cargorobot, il sonar, il markerdevice e l'IOPort saranno rappresentati tramite collaboratori simulati.

Proposta di aggiunta: =Evoluzione dal modello dello sprint 0

Prendendo come punto di partenza il modello sviluppato alla fine dello sprint0, dobbiamo poi ecc.. ecc..

2. Requirements

I requisiti del progetto sono riportati nel documento disponibile al seguente [link](#)^o

L'azienda richiede di realizzare un servizio denominato **cargoservice** con il seguente funzionamento:

- Il cargoservice è in grado di ricevere una richiesta di carico di un container inviata da un cliente tramite il pulsante dell'IOPort.
- Invia la risposta **retrylater** se l'IOPort è attualmente occupato da un container oppure se il sistema è **Out of service**.
- Rifiuta la richiesta quando la hold è già piena, ovvero gli slot1-4 sono già tutti occupati.
- Altrimenti, considera il sistema come **engaged**, rileva uno slot libero e restituisce come risposta il nome dello slot riservato. Mentre il sistema è engaged, il LED deve lampeggiare.
- Quando la richiesta di carico viene accettata, il cliente deve spostare il container nell'area del sensore entro un tempo prefissato (ad esempio 30 secondi), altrimenti il sistema diventa **disengaged**.

- Successivamente, il cargoservice utilizza il cargorobot per spostare il container dall'IOPort a slot5 (per l'etichettatura del container) e poi allo slot riservato.
- Il servizio deve inoltre mostrare sul display dell'IOPort:
 - lo stato attuale della hold
 - il messaggio “**Service working**” quando tutto sta procedendo correttamente
 - il messaggio “**Out of service**” se il sensore sonar misura una distanza $D > D_{FREE}$ per almeno 3 secondi (possibile guasto del sonar)

3. Requirement analysis

L'analisi dettagliata del dominio e dei requisiti è già stata affrontata nello Sprint 0, in questa fase ci concentriamo esclusivamente sul ciclo di gestione delle richieste di carico (**core business**) del sistema.

Lo svolgimento corretto di tali azioni presuppone tuttavia dell'esistenza di altri componenti del sistema fisici o in forma simulata (**sonar**, **LED**, **cargorobot**, **markerdevice**, **IOPort** come web GUI) non ancora sviluppati. La loro realizzazione concreta è pianificata per gli sprint successivi. I componenti “mock” che replicheranno, per test, il comportamento di quelli mancanti verranno evoluti rispetto alla formalizzazione QAK già avvenuta in fase di Sprint0 (recuperabile al seguente [link](#)^o).

4. Problem analysis

Come emerso dall'analisi dei requisiti, questo componente funge da **orchestratore**: coordina le operazioni degli altri componenti del sistema al fine di eseguire le procedure di carico. Inoltre, le entità sono distribuite su quattro nodi separati ma, per non rallentare la prototipazione, tutti i componenti verranno rappresentati nello stesso nodo (rappresentato da un Context), tenendo in considerazione che gli attori non condividono memoria, e comunicano tra loro tramite scambio di messaggi.

4.1. Rappresentazione dello stato interno della stiva

In questa fase la gestione della stiva non viene ancora implementata completamente. Tuttavia, per rispettare il principio di singola responsabilità, la logica non viene inclusa nel cargoservice, ma viene delegata a un **attore mock autonomo** posizionato nel context ctxdevices

```
QActor hold context ctxdevices {
  [#
    var Slots = intArrayOf(0, 0, 0, 0)

    fun getFreeSlot(): Int {
      for (i in 0..3) {
        if (Slots[i] == 0) return i + 1
      }
      return -1
    }
  #]
```

```
// ...
}
```

Ogni posizione dell'array rappresenta uno slot della stiva. Il valore `0` indica uno slot libero, mentre un valore diverso da `0` indica uno slot occupato o riservato.

Questa scelta permette al **cargoservice** di verificare dinamicamente se la stiva è piena interrogando l'attore hold tramite Request / Reply, senza conoscerne i dettagli interni

4.2. Analisi delle Interazioni

Di seguito si riporta una formalizzazione delle altre interazioni modellate interpretando e applicando opportune scelte progettuali sui requisiti del sistema.

- Sappiamo che il cargoservice deve interrogare la hold per sapere se c'è un posto libero ed eventualmente riservarlo. Il cargoservice non può quindi procedere finché la hold non ha risposto in modo affermativo (in caso di slot libero) o negativo (in caso di stiva piena). Scegliamo quindi una comunicazione di tipo Request/Reply.
- Sarà anche necessario fornire un meccanismo per liberare lo slot una volta che il timeout di 30 secondi sarà scaduto. In questo caso, non è necessario attendere una risposta, quindi si utilizza una Dispatch.

```
// CargoService ↔ Hold
Request get_slot      : getSlot(none)
Reply  slot_reserved  : slotReserved(SLOTID) for get_slot
Reply  hold_full      : holdFull(none) for get_slot
Dispatch free_slot    : freeSlot(SLOTID) // NON NECESSARIO, guarda risposta
committente sprint0
```

- Da requisiti, sappiamo che il sonar deve misurare continuamente la distanza del container dal sonar stesso. Nasce quindi la necessità di dover trasmettere queste misurazioni al cargoservice. Per isolare la responsabilità del sonar a semplice “misuratore e trasmettitore” di informazione, viene naturale formalizzare tale comunicazione come un Event, ovvero un messaggio broadcast che verrà ascoltato da chi interessato (in questo caso **cargoservice**)

```
Event    sonardata      : distance(D)
```

- L'invio di comandi al led è delegato al cargoservice. Non è necessario attendere una risposta, quindi utilizziamo una Dispatch.

```
Dispatch led_ctrl      : ledCmd(CMD)
// CMD: "on", "off", "blink"
```

- In questa fase, il movimento del robot sarà simulato, ma il cargoservice deve comunque attendere la fine dell'operazione per poter procedere con la

successiva fase del ciclo operativo (ossia richiedere al **markerdevice** l'etichettatura in **slot5**, confermare lo stoccaggio e tornare allo stato **disengaged**). Usiamo quindi la Request/Reply per assicurarci che il cargoservice attenda la fine del task.

```
// CargoService ↔ Robot
Request robot_move : robotMove(TARGET)
Reply  robot_done  : robotDone(none) for robot_move
```

- In modo analogo, il cargoservice deve interrogare il markerdevice per sapere quando l'operazione di etichettatura è terminata, non potendo procedere finché il markerdevice non ha risposto. Usiamo quindi la Request/Reply.

```
// CargoService ↔ MarkerDevice
Request mark_container : markContainer(none)
Reply  marking_done   : markingDone(none) for mark_container
```

Analisi del Blocco 1: La prima sezione del modello definisce i confini di interazione e l'infrastruttura di comunicazione.

- **Distinzione semantica dei messaggi:** In accordo con le buone pratiche di ingegneria dei protocolli, si distingue rigorosamente tra comunicazioni binarie sincrone/correlate (**Request** / **Reply**), comandi asincroni diretti (**Dispatch** , utilizzati ad esempio per liberare lo slot con **free_slot** o per accendere/spegnere il LED con **led_ctrl**) e notifiche broadcast ambientali (**Event** , impiegato dal sonar per diffondere la misura della distanza **sonardata**).
- **Contesto Unico:** La dichiarazione di **ctxprototype** sulla porta **8050** vincola tutti i 7 attori del sistema a eseguire nella stessa JVM. Come argomentato nell'analisi, ciò elimina il rumore infrastrutturale e le latenze TCP durante i test di correttezza della business logic.

4.3. Vocabolario delle Interazioni

- Per quanto riguarda l'interazione tra **ioportmock** e **cargoservice** si utilizzano i messaggi già

definiti in fase di Sprint 0:

```
Request load_request      : loadRequest(none)
Reply load_accepted      : loadAccepted(slotID) for load_request
Reply load_retrylater    : loadRetryLater(none) for load_request
Reply load_refused       : loadRefused(none) for load_request
```

- Per quanto riguarda l'interazione tra Cargoservice e Cargorobot e Markerdevice (simulati rispettivamente da **cargorobotmock** e **markerdevice**) si considera che, una volta accettato il carico, il cargoservice deve delegare le operazioni fisiche di pesatura/etichettatura al **markerdevice** e di trasporto al **cargorobotmock**. L'orchestratore deve sapere esattamente quando queste operazioni asincrone termi-

nano per poter aggiornare lo stato del sistema. Usiamo la Request/Reply per assicurarci che il cargoservice attenda la fine del task

```
// Gestione Marker
Request mark_container : markContainer(none)
Reply  marking_done   : markingDone(none) for mark_container

// Gestione Robot
Request robot_move : robotMove(TARGET) // TARGET: "slot5", "slot1", ecc.
Reply  robot_done  : robotDone(none) for robot_move
```

- Il **cargoservice** deve interrogare la **hold** per sapere se c'è un posto libero ed, eventualmente, riservarlo. Anche in questo caso, il cargoservice non può procedere finché la stiva non ha risposto. Scegliamo quindi il pattern Request/Reply:

```
Request get_slot      : getSlot(none)
Reply  slot_reserved  : slotReserved(SLOTID) for get_slot
Reply  hold_full      : holdFull(none) for get_slot
Dispatch free_slot    : freeSlot(SLOTID)
```

- Il sensore fisico (simulato dal **sonarmock**) rileva continuamente la distanza e non conosce a priori quali componenti del sistema siano interessati a questo dato. Il Sonar emette un Event, ovvero un messaggio broadcast (con destinatario implicito):

```
Event sonardata : distance(D) // Emesso dal sonar nell'ambiente
```

4.4. Rappresentazione dell'attore cargoservice come Macchina a Stati Finiti

L'attore cargoservice è, già da Sprint 0, inteso come una Macchina a Stati Finiti che utilizza variabili interne per mantenere la conoscenza dello stato applicativo:

```
[#
  var IOPortOccupied = false
  var ServiceWorking = true
  var CargoState     = "disengaged"
  var ReservedSlotId = -1
#]
```

Si gestisce quindi il ciclo di vita della richiesta seguendo le transizioni principali:

Proposta di aggiunta: descrizione dettagliata in linguaggio naturale del funzionamento del codice QAK

```
QActor cargoservice context ctxcargoservice {
  // ... inizializzazione variabili ...

  State disengaged { }
  Transition t0
```



```

whenRequest load_request      → handle_load_request
whenEvent   sonardata         → handle_sonar
whenMsg     set_service_status → update_service

State handle_load_request {
  if [# CargoState = "engaged" || !ServiceWorking || IOPortOccupied #] {
    replyTo load_request with load_retrylater : loadRetryLater(none)
  } else {
    request hold -m get_slot : getSlot(none)
  }
}
Goto returnToState if [# CargoState = "engaged" || !ServiceWorking ||
IOPortOccupied #] else wait_for_slot

State wait_for_slot {}
Transition t0
  whenReply slot_reserved → accept_request
  whenReply hold_full    → refuse_request

State accept_request {
  onMsg(slot_reserved : slotReserved(ID)) {
    [# ReservedSlotId = payloadArg(0).toInt(); CargoState = "engaged" #]
    forward ledmock -m led_ctrl : ledCmd(blink)
    [# val SlotName = "slot$ReservedSlotId" #]
    replyTo load_request with load_accepted : loadAccepted($SlotName)
  }
}
Goto engaged

State handle_sonar {
  onMsg(sonardata : distance(D)) {
    [# IOPortOccupied = (payloadArg(0).toInt() < 50) #]
  }
}
Goto do_robot_job if [# IOPortOccupied && CargoState = "engaged" #] else
returnToState

// ... logica robot_job e timeout ...
}

```

5. Test plans

Al fine di validare il core-business implementato in questo Sprint, il piano di testing automatizzato (sviluppato in JUnit e Kotlin) si concentra sul verificare che il `cargoservice` rispetti il protocollo di Request/Reply e aggiorni correttamente il suo stato interno al variare delle condizioni simulate.

I test operano instradando messaggi `TCP` in formato stringa-Prolog sulla porta `8050` (dove è in ascolto il `cargoservice`) e verificandone le risposte.

Di seguito il codice sorgente del Test Plan implementato:

```

package it.unibo.cargoservice

import org.junit.Assert.assertTrue
import org.junit.Assert.fail
import org.junit.After
import org.junit.Before
import org.junit.Test
import unibo.basicomm23.interfaces.Interaction
import unibo.basicomm23.tcp.TcpClientSupport
import unibo.basicomm23.utils.CommUtils

class TestCargoServiceCore {
    private var conn: Interaction? = null

    @Before
    fun setup() {
        CommUtils.outcyan(" == [TestCargoServiceCore] SETUP: Connecting to CargoService == ")
        try {
            // Connessione al contesto del cargoservice sulla porta 8050
            conn = TcpClientSupport.connect("127.0.0.1", 8050, 10)
        } catch (e: Exception) {
            fail("Connessione TCP fallita.")
        }
    }

    @After
    fun teardown() {
        conn?.close()
    }

    @Test
    fun test01_LoadAccepted() {
        CommUtils.outmagenta(" --- test01_LoadAccepted --- ")
        try {
            // Formato msg: msg(MSGID, MSGTYPE, SENDER, RECEIVER, CONTENT, SEQNUM)
            val req = "msg(load_request, request, testunit, cargoservice, loadRequest(none), 1)"

            // Invio request e attesa sincrona della reply
            val reply = conn?.request(req)

            // Verifica: il sistema doveva accettare la richiesta
            assertTrue(reply != null && reply.contains("load_accepted"))
        } catch (e: Exception) {
            fail("Test fallito: ${e.message}")
        }
    }

    @Test
    fun test02_LoadRetryLater_OutOfService() {
        CommUtils.outmagenta(" --- test02_LoadRetryLater_OutOfService --- ")
        try {
            // 1. Forzatura del sistema in Out Of Service (simulazione

```

```

sonar guasto)
    val dispatchFault = "msg(set_service_status, dispatch, testunit,
cargoservice, setServiceStatus(outofservice), 2)"
    conn?.forward(dispatchFault)
    Thread.sleep(500)

    // 2. Invio request mentre il sistema è in guasto
    val req = "msg(load_request, request, testunit, cargoservice,
loadRequest(none), 3)"
    val reply = conn?.request(req)

    // 3. Verifica: il sistema doveva rifiutare temporaneamente
la richiesta
    assertTrue(reply != null && reply.contains("load_retrylater"))

    // 4. Ripristino del sistema
    val dispatchRecover = "msg(set_service_status, dispatch,
testunit, cargoservice, setServiceStatus(working), 4)"
    conn?.forward(dispatchRecover)
    Thread.sleep(500)
} catch (e: Exception) {
    fail("Test fallito: ${e.message}")
}
}
}

```

6. Deployment

Il deployment del prototipo di Sprint 1 consiste nell'avvio dei singoli progetti che compongono l'architettura distribuita del sistema.

La directory `prototype` contiene i seguenti progetti:

- `cargoservice` °
- `customer` °
- `devices` °
- `robot` °

Ogni progetto deve essere compilato e avviato separatamente tramite Gradle.

Per ciascuna directory è sufficiente eseguire:

```

./gradlew build
./gradlew run

```

7. Pagina di sintesi

7.1. Architettura finale del prototipo

L'architettura finale del prototipo sviluppato durante questo Sprint è riportata nella figura seguente.

Per maggiori dettagli sul modello implementato si rimanda a [ProblemAnalysis](#), mentre i test sviluppati sono descritti in [Test Plan](#).

Questa architettura rappresenta il risultato finale dello Sprint 1 e costituirà il punto di partenza per lo Sprint 2.

7.2. Verifica della pianificazione

La pianificazione prevista per lo Sprint 1 è stata rispettata. Non si sono verificati scostamenti significativi rispetto agli obiettivi inizialmente prefissati.

7.3. Goal dello Sprint 2

L'obiettivo principale dello Sprint 2 sarà incrementare il livello di realismo del prototipo sostituendo i componenti simulati con le rispettive implementazioni reali.

In particolare, le attività previste sono:

- sostituire il **cargorobotmock** con l'attore reale interfacciato a **VirtualRobot26**;
- realizzare l'**IOPort** come Web GUI, consentendo l'interazione dell'utente tramite browser;
- integrare i nuovi componenti mantenendo invariata l'architettura ad attori del sistema;
- verificare il corretto funzionamento del sistema mediante un aggiornamento dei test funzionali.

8. Team di lavoro

NOME E COGNOME	MATRICOLA
Davide Chirichella	0001222371
Gabriele Doti	0001245897
Daniele Maccagnan	0001247340